

Better mdspan's CTAD

- - Abstract
 - Revision history
 - Discussion
 - Design
 - Proposed change
 - References

Abstract

This paper makes `span` and `mdspan`' CTAD `integral_constant`-aware to deduce more efficient types.

Revision history

R0

Initial revision.

R1

1. Changed the comma in CTAD of `span(It, End0rSize)`, which is part of the wording, to green.
2. Initialized the *maybe-static-ext*'s specialization with `{T::value}` to prevent conversions from negative values.
3. Aligned with the latest working draft.

Discussion

Currently, `mdspan`'s most common pointer-indices CTAD have the following definitions:

```
template<class ElementType, class... Integrals>
requires((is_convertible_v<Integrals, size_t> && ...) && sizeof...(Integrals) > 0)
explicit mdspan(ElementType*, Integrals...)
-> mdspan<ElementType, dextents<size_t, sizeof...(Integrals)>>;
```

Since the `Extents` template parameter of `mdspan` is explicitly specified as `dextents`, the deduced type always has dynamic extent, even if we pass a compile-time constant:

```
mdspan ms (p, 3, 4, 5); // mdspan<int, extents<size_t, dynamic_extent, dynamic_extent, dynamic_extent>>
mdspan ms2(p, 3, integral_constant<size_t, 4>{}, 5); // ditto
mdspan ms3(p, integral_constant<size_t, 3>{}, 4, integral_constant<size_t, 5>{}); // ditto
```

This result feels wrong to me. If `integral_constant` represents a compile-time constant, why don't we take advantage of it and make it reflect the extent of `mdspan`, since we already did this with `strided_slice`?

The author believes that instead of setting static extents by additionally creating `extents`:

```
mdspan ms2(p, extents<size_t, dynamic_extent, 4, dynamic_extent>(3, 5)); // mdspan<int, extents<size_t, dynamic_extent, 4, dynamic_extent>>
mdspan ms3(p, extents<size_t, 3, dynamic_extent, 5>(4)); // mdspan<int, extents<size_t, 3, dynamic_extent, 5>>
```

It would be better to automatically make the first example equivalent based on whether the argument is a compile-time constant.

This brings the following benefits:

1. It enables defining dynamic or static extents with the similar form and intuitively reflects the extent type.
2. It's less error-prone as we don't need to calculate how many `dynamic_extents` there are to pass in the correct number of arguments. The compiler already handle it for us.
3. It makes defining `mdspan` more concise, an `mdspan` with mixed extents now can be spelled as `mdspan(c_<3>{}, 4, c_<5>{})` (benefit from [P2781](#) `std::constexpr_v`).

Design

`span`'s CTAD

The paper also applies to `span`' CTAD for consistency. In other words, `span(p, c_<5>{})` will be deduced as `span<int, 5>`. The author doesn't think this is a big issue as it's relative intuitive and generally users don't often write this way.

`mdspan`'s CTAD

Given that [P2630](#) `std::submdspan` already introduced *integral-constant-like* for checking `integral_constant`-like type, this paper reuses it to detect whether `mdspan`'s arguments are a compile-time constant. This also applies to `extents`.

Proposed change

This wording is relative to [N4971](#).

1. Edit 24.7.2.1 [\[span.syn\]](#) as indicated:

```
namespace std {
    // constants
    inline constexpr size_t dynamic_extent = numeric_limits<size_t>::max();

    template<class T>
    concept integral_constant_like = // exposition only
    is_integral v<decltype(T::value)> &&
    !is_same v<bool, remove_const_t<decltype(T::value)>> &&
    convertible_to<T, decltype(T::value)> &&
    equality_comparable_with<T, decltype(T::value)> &&
    bool_constant<T() == T::value::value &&
    bool_constant<static_cast<decltype(T::value)>(T()) == T::value::value;

    template<class T>
    constexpr size_t maybe_static_ext = dynamic_extent; // exposition only
    template<integral_constant_like T>
    constexpr size_t maybe_static_ext<T> = {T::value};

    // [views.span], class template span
    template<class ElementType, size_t Extent = dynamic_extent>
    class span;
    [...]
}
```

2. Edit 24.7.2.2.1 [\[span.overview\]](#) as indicated:

-2- All member functions of span have constant time complexity.

```
namespace std {
    template<class ElementType, size_t Extent = dynamic_extent>
    class span {
        [...]
    };

    template<class It, class EndOrSize>
    span(It, EndOrSize) -> span<remove_reference_t<iter_reference_t<It>>, maybe_static_ext<EndOrSize>>;
    [...]
}
```

3. Edit 24.7.2.2.3 [\[span.deduct\]](#) as indicated:

```
template<class It, class EndOrSize>
span(It, EndOrSize) -> span<remove_reference_t<iter_reference_t<It>>, maybe_static_ext<EndOrSize>>;
```

-1- Constraints: It satisfies `contiguous_iterator`.

4. Edit 24.7.3.3.3 [\[mdspan.extents.cons\]](#) as indicated:

```
template<class... Integrals>
explicit extents(Integrals...) -> see below;
```

-1- Constraints: `(is_convertible_v<Integrals, size_t> && ...)` is true.

-2- Remarks: The deduced type is `dextents<size_t, maybe_static_ext<Integrals>...sizeof...(Integrals)>`.

5. Edit 24.7.3.2 [\[mdspan.syn\]](#) as indicated:

```
namespace std {
    [...]
    template<class T>
    concept integral_constant_like = // exposition only
    is_integral v<decltype(T::value)> &&
    !is_same v<bool, remove_const_t<decltype(T::value)>> &&
    convertible_to<T, decltype(T::value)> &&
    equality_comparable_with<T, decltype(T::value)> &&
    bool_constant<T() == T::value::value &&
    bool_constant<static_cast<decltype(T::value)>(T()) == T::value::value;
    [...]
}
```

6. Edit 24.7.3.6.1 [\[mdspan.mdspan.overview\]](#) as indicated:

-1- `mdspan` is a view of a multidimensional array of elements.

```
namespace std {
    template<class ElementType, class Extents, class LayoutPolicy = layout_right,
            class AccessorPolicy = default_accessor<ElementType>>
    class mdspan {
        [...]
    };

    template<class CArray>
    requires(is_array_v<CArray> && rank_v<CArray> == 1)
    mdspan(CArray&)
    -> mdspan<remove_all_extents_t<CArray>, extents<size_t, extent_v<CArray, 0>>>;
}
```

```

template<class Pointer>
requires(is_pointer_v<remove_reference_t<Pointer>>)
mdspan(Pointer&&)
-> mdspan<remove_pointer_t<remove_reference_t<Pointer>>, extents<size_t>>;

template<class ElementType, class... Integrals>
requires((is_convertible_v<Integrals, size_t> && ...) && sizeof...(Integrals) > 0)
explicit mdspan(ElementType*, Integrals...)
-> mdspan<ElementType, dextents<size_t, maybe-static-ext<Integrals>..., sizeof...(Integrals)>>;

[...]
```

References

[P2630R4]

Christian Trott. Submdspan. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2630r4.html>

[P2781R3]

Matthias Kretz. std::constexpr_v. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2781r3.html>